

Projektdokumentation

SE-GL ABSCHLUSSPROJEKT

Matthias
LUG |

EINLEITUNG	2
PROJEKT INFORMATIONEN	2
KONKRETE PROJEKT BESCHREIBUNG	2
PROJEKTANTRAG / PLANUNG	3
PFLICHTFUNKTIONEN.....	3
OPTIONALE FEATURES.....	3
GEPLANTE ARCHITEKTUR	4
EINSATZ VON NICHT-STANDARD-BIBLIOTHEKEN	5
INHALT DER PROJEKTDOKUMENTATION	5
ZEITPLANUNG / PROJEKTTABLAUF	6
GANTT DIAGRAMM	6
ARCHITEKTUR	7
KLASSEN UND METHODEN() PLAN	7
CODESNIPPETS.....	11
PROGRAMMABLAUF	17
ERKLÄRUNG DER BENUTZER OBERFLÄCHE	17
FLOWCHART.....	24
IMPLEMENTIERUNG UND UMSETZUNG.....	25
SCHRITTWEISE IMPLEMENTIERUNG	25
TEST UND DEBUGGING-VORGEHEN	27
BEKANNTE FEHLER.....	27
LÖSUNGSANSÄTZE UND DESIGNENTSCHEIDUNGEN	28
REFLEKTION	29
REFLEKTION DER PLANUNG	29
REFLEKTION DER UMSETZUNG.....	29
REFLEKTION DER DOKUMENTATION.....	30
PERSÖNLICHE REFLEKTION	30
QUELLENANGABEN	30

Einleitung

Projekt Informationen

Umschüler: Matthias

GitHub-Link: <https://github.com/Lord-Mandos>

Ausbildungsstätte/Ausbildungsfirma/Firma: LuG

Projektbezeichnung: Konsolenbasiertes Aufgaben- & Projektmanagement-Tool („Kaban“)

Zulassender Dozent: Jochen

Bewertender Dozent: Michael

Konkrete Projektbeschreibung

„Taskhub“ ist eine Konsolenanwendung zur strukturierten Verwaltung von Aufgaben innerhalb kleiner Projekte.

Das Tool soll Anwendern ermöglichen, Aufgaben zu planen, zu organisieren und den Fortschritt zu kontrollieren.

Die Software orientiert sich an einfachen Projektmanagement-Methoden wie Kanban und To-Do-Listen.

Ziel ist es, eine klare, intuitive Konsolenoberfläche zu schaffen, mit der ein Benutzer Aufgaben:

- *erstellen,*
- *bearbeiten,*
- *löschen*
- *und in verschiedene Statusbereiche verschieben kann*

Zur Verbesserung der Sicherheit wird die Benutzerverwaltung mit *PBKDF2-Passwort-Hashing* umgesetzt.

Die Oberfläche wird mit *Spectre.Console* gestaltet, um eine moderne und strukturierte Darstellung im Terminal zu ermöglichen (z. B. Tabellen, Panels, farbige Hervorhebungen).

Alle Daten werden lokal in JSON-Dateien gespeichert und beim Programmstart wieder geladen.

Projektantrag / Planung

Pflichtfunktionen

Benutzerverwaltung

- ❖ *Benutzer registrieren*
- ❖ *Anmeldung über Benutzername + Passwort*
- ❖ *Speicherung von:*
 - *Passwort-Hash (PBKDF2)*
 - *Salt*
 - *Iterationsanzahl*
- ❖ *Kein Klartextspeichern*

Aufgabenmanagement

- ❖ *Aufgaben erstellen*
- ❖ *Aufgaben anzeigen*
- ❖ *Aufgaben bearbeiten*
- ❖ *Aufgaben löschen*
- ❖ *Aufgaben suchen (ID, Titel, Kategorie)*

Kanban-Board

- ❖ *Statuswechsel zwischen Backlog, In Arbeit, Erledigt*
- ❖ *Darstellung mit Spectre.Console (Tabellen/Layouts)*

Datenspeicherung

- ❖ *JSON-Dateien für Benutzer- und Aufgabendaten*
- ❖ *Automatisches Speichern nach Änderungen*
- ❖ *Laden beim Programmstart*

Spectre.Console UI

- ❖ *Tabellen für Kanban-Board*
- ❖ *Panels, Markup und Farben*
- ❖ *Auswahlmenüs (SelectionPrompt)*

Optionale Features

- ❖ *Statistiken (z. B. erledigte Aufgaben je Zeitraum)*
- ❖ *Deadline-Verwaltung*
- ❖ *CSV-/TXT-Export*
- ❖ *Prioritätsmarkierungen (Farben)*
- ❖ *Benutzerrollen (Admin/User)*

Geplante Architektur

Klassen (geplant)

❖ User

- *ID*
- *Username*
- *PasswordHash*
- *Salt*
- *Tasks*

❖ Task

- *ID*
- *Title*
- *Description*
- *Category*
- *Priority*
- *Status*
- *CreatedAt*
- *CompletedAt*

❖ AuthService

- *PasswordHashWithPBKDF2()*
- *VerifyPassword()*

❖ TaskService

- *AddTask()*
- *EditTask()*
- *DeleteTask()*
- *MoveTask()*
- *SearchTasks()*

❖ StorageManager

- *Save()*
- *Load()*

❖ MenuSystem

- *LoginMenu()*
- *MainMenu()*
- *TaskMenu()*
- *BoardMenu()*

❖ UIRenderer (für Spectre.Console)

- *RenderBoard()*
- *RenderTaskList()*
- *RenderMessage()*

Einsatz von Nicht-Standard-Bibliotheken

Für die Umsetzung des Projekts werden folgende nicht zum üblichen Standardumfang gehörenden Bibliotheken verwendet:

System.Security.Cryptography –(PBKDF2)

Wird verwendet, um Passwörter sicher mit PBKDF2 zu hashen.

Dies dient der sicheren Authentifizierung und verhindert die Speicherung von Klartextpasswörtern.

Spectre.Console

Diese Bibliothek wird zur Gestaltung einer modernen und strukturierten Konsolenoberfläche eingesetzt.

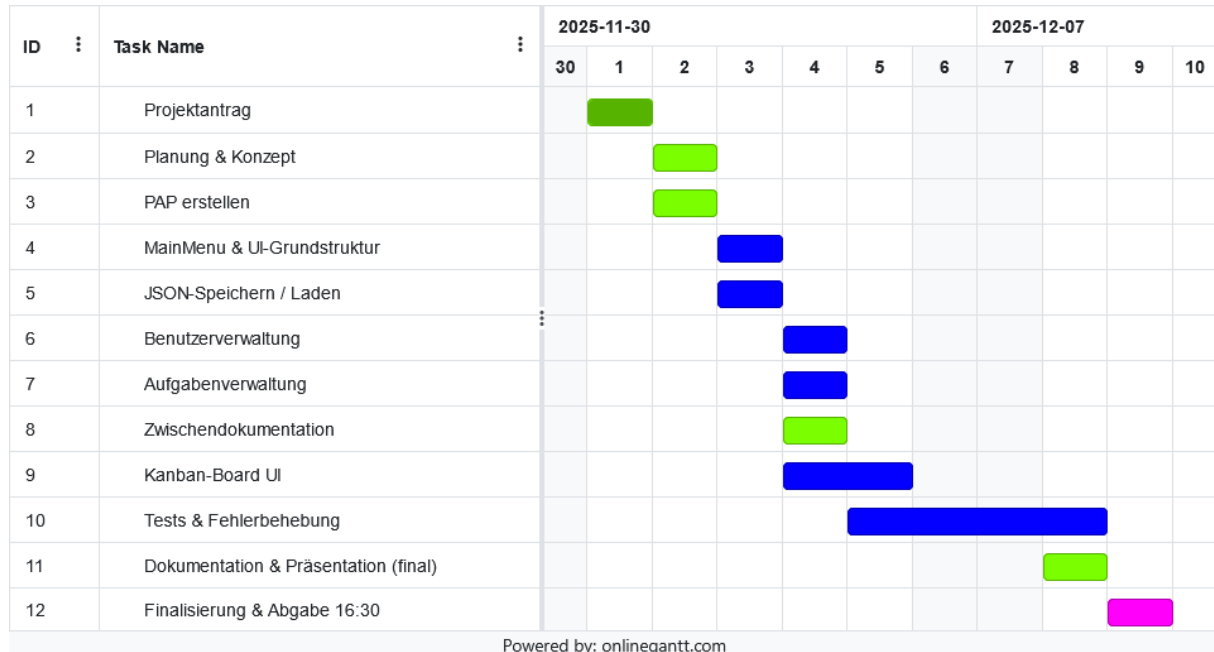
Damit können Tabellen, Panels, Markup-Text, Auswahlmenüs und farbliche Hervorhebungen realisiert werden, um das Kanban-Board und die Navigation übersichtlich darzustellen.

Inhalt der Projektdokumentation

1. *Einleitung*
2. *Projektantrag / Planung*
3. *Zeitplanung / Projektablauf*
4. *Implementierung und Umsetzung*
5. *Probleme, Schwierigkeiten und Überarbeitungen*
6. *Screenshots (Code & Debugging)*
7. *Diagramme (Flowchart, Nassi-Schneiderman)*
8. *Quellenangaben*
9. *Anlagen*

Zeitplanung / Projektablauf

Gantt Diagramm



Erläuterung zum Gantt Diagramm

Nach Genehmigung des Projektantrages wird das Projekt geplant und ein PAP erstellt, um eine stabile Basis zu schaffen.

Die UI-Grundstruktur und das JSON-Speichersystem wurden früh eingeplant, damit alle späteren Module darauf aufbauen können.

Darauf werden Benutzerverwaltung und Aufgabenverwaltung implementiert, womit die Grundfunktionen des Programmes funktionsfähig sind.

Durch die Zwischendokumentation am 04.12. wird die Enddokumentation deutlich entlastet.

Die Kanban-Board UI wird zu Schluss eingeplant da sie auf allen anderen Funktionen aufbaut.

Tests und Fehlerbehebung wurden bewusst über mehrere Tage verteilt und bilden den Übergang zur finalen Dokumentation.

Der letzte Tag ist ausschließlich für finale Anpassungen, Aufräumen und die Abgabe reserviert.

Architektur

Klassen und Methoden() plan

TaskItem

- **Felder/Eigenschaften:**
 - Guid Id
 - string Title
 - string Description
 - DateTime CreateAt
 - DateTime DueDate
 - TaskState Status
 - string AssignedUser

User

- **Felder/Eigenschaften:**
 - string Username
 - UserRole Role
 - string PasswordHash
 - byte[] Salt
- **Methoden:**
 - void SetPassword()
 - bool ValidatePassword()

Enums

- **TaskState:** ToDo, InProgress, Done
- **UserRole:** User, Admin

StorageManager

- **Methoden:**
 - static List<T> Load(string filePath)
 - static void Save(string filePath, List<T> data)

TaskRepository

- **Felder:**
 - const string FilePath
- **Methoden:**
 - List<TaskItem> LoadAllTasks()
 - List<TaskItem> LoadTasks()
 - void SaveTasks(List<TaskItem> tasks)
 - void SaveAllTasks(List<TaskItem> allTasks)

UserRepository

- **Felder:**
 - const string FilePath
- **Methoden:**
 - List<User> LoadUsers()
 - void SaveUsers(List<User> users)

TaskService

- **Methoden:**
 - void CreateTask()
 - void DeleteTask()
 - void UpdateTask()
 - void ShowTasks()
 - void ShowKanbanBoard()
 - void ChangeTaskStatus()
 - void ShowTaskDetails()

UserService

- **Methoden:**
 - void CreateUserAdmin()
 - void EnsureInitialAdmin()
 - void CreateUser()
 - void GetUsers()
 - void UpdateUser()
 - void DeleteUser()

PasswordHelper

- **Methoden:**
 - static byte[] GenerateSalt(int size = 16)
 - static string HashPassword(string password, byte[] salt)
 - static bool VerifyPassword(string password, byte[] salt, string passwordhash)

AuthManager

- **Eigenschaften:**
 - User? LoggedInUser
- **Methoden:**
 - bool Login()
 - void Logout()
 - bool IsAdmin()

Session

- **Eigenschaften:**
 - User? CurrentUser
- **Methoden:**
 - void StartSession()

MenuSystem

- **Felder:**
 - static IReadOnlyList<Markup> MainMenuText
 - static IReadOnlyList<Markup> TaskMenuText
 - static IReadOnlyList<Markup> UserMenuText
 - static IReadOnlyList<Markup> KanbanBoardMenuText
 - static IReadOnlyList<Markup> StartMenu
- **Methoden:**
 - static Panel MenuPanel(IReadOnlyList<Markup> menuText, string menuTitle)
 - static void UpdateMainOverview(string action)
 - static void UserMenuChoice(IReadOnlyList<Markup> menuText)

UIRenderer

- **Methoden:**
 - static void UIMain(IReadOnlyList<Markup> menuText, string menuTitle)
 - static void Refresh(IReadOnlyList<Markup> menuText, string menuTitle)

BodyRightManager

- **(Statische Felder für Inhalt und Titel)**
- **Methoden:**
 - static void SetTitle(string title)
 - static void Set(string text)
 - static void Add(string text)
 - static void SetRenderable(IRenderable renderable)
 - static void Clear()
 - static Panel GetPanel()

HeaderFooterManager

- **Methoden:**
 - static Panel GetHeaderLeft()
 - static Panel GetHeaderRight()
 - static Panel GetFooter()

CodeSnippets

Hier sind die wichtigsten Funktionen als Snippet mit einer Wofür und Warum Erklärung

Eindeutige Task-Titel prüfen

Klasse: TaskService

Methode: CreateTask

```
newTask.Title = AnsiConsole.Prompt<string>(  
    new TextPrompt<string>("[bold yellow]Aufgaben Titel eingeben:[/]")  
    .PromptStyle("green")  
    .Validate(title =>  
    {  
        var t = (title ?? string.Empty).Trim();  
        if (t.Length < 3)  
            return ValidationResult.Error("[red]Der Titel muss mindestens 3 Zeichen lang sein.[/]");  
        if (tasks.Any(x => x.Title.Equals(t, StringComparison.OrdinalIgnoreCase)))  
            return ValidationResult.Error("[red]Ein Task mit diesem Titel existiert bereits. Bitte wählen Sie  
einen eindeutigen Titel.[/]");  
        return ValidationResult.Success();  
    }  
));
```

Wofür: Der User darf nur eindeutige, mindestens 3 Zeichen lange Titel vergeben.

Warum: Verhindert Dubletten, hält die Aufgabenliste sauber.

Passwort-Hashing mit PBKDF2

Klasse: PasswordHelper

Methode: HashPassword

```
public static string HashPassword(string password, byte[] salt)
{
    var hash = Rfc2898DeriveBytes.Pbkdf2(
        password,
        salt,
        10000,
        HashAlgorithmName.SHA256,
        32
    );
    return Convert.ToBase64String(hash);
}
```

Wofür: Passwort-Hash mit Salt und 10.000 Iterationen speichern.

Warum: Standard für sichere Passwortspeicherung.

Eigene Paginierung in der Benutzeranzeige

Klasse: UserService

Methode: GetUsers

```
const int pageSize = 12;
int page = 0;
int pages = (users.Count + pageSize - 1) / pageSize;

while (true)
{
    var pageUsers = users.Skip(page * pageSize).Take(pageSize).ToList();
    // ... Anzeige und Navigation hier ...
    var actions = new List<string>();
    if (page > 0) actions.Add("← Zurück");
    if (page < pages - 1) actions.Add("Weiter →");
    actions.Add("Zurück zum Menü");

    var choice = AnsiConsole.Prompt(
        new SelectionPrompt<string>()
            .Title("Wähle Aktion:")
            .AddChoices(actions));

    if (choice == "Weiter →") { page++; continue; }
    else if (choice == "← Zurück") { page--; continue; }
    else { break; }
}
```

Wofür: Zeigt Benutzer seitenweise an, steuert durch Button-Auswahl.

Warum: Übersicht bei vielen Nutzern.

Kanban-Spalten aus echten Tasks bilden

Klasse: TaskService

Methode: ShowKanbanBoard

```
var todo = tasks.Where(t => t.Status == TaskState.ToDo).ToList();
var inProgress = tasks.Where(t => t.Status == TaskState.InProgress).ToList();
var done = tasks.Where(t => t.Status == TaskState.Done).ToList();
```

Wofür: Sortiert Aufgaben für die Kanban-Ansicht nach Status.

Warum: Grundlage für strukturierte Board-Ansicht.

Passwort-Eingabe & Validierung am User

Klasse: User

Methode: SetPassword und ValidatePassword

```
public void SetPassword()
{
    var password = AnsiConsole.Prompt<string>(
        new TextPrompt<string>("Bitte geben Sie Ihr Passwort ein:")
        .PromptStyle("red")
        .Secret());
    Salt = PasswordHelper.GenerateSalt();
    PasswordHash = PasswordHelper.HashPassword(password, Salt);
}

public bool ValidatePassword()
{
    var password = AnsiConsole.Prompt<string>(
        new TextPrompt<string>("Bitte geben Sie Ihr Passwort ein:")
        .PromptStyle("red")
        .Secret());
    return PasswordHelper.VerifyPassword(password, this.Salt, this.PasswordHash);
}
```

Wofür: Holt Passwort verdeckt vom User, hasht und prüft es.

Warum: Privatheit und Sicherheit bei Nutzeranmeldung.

Nur Admin darf zur Benutzerverwaltung

Klasse: MenuSystem

Ausschnitt in UserMenuChoice

```
if (Session.CurrentUser != null && Session.CurrentUser.Role == UserRole.Admin)
{
    UpdateMainOverview("Benutzerverwaltung geöffnet");
    UIRenderer.UIMain(UserMenuText, "Benutzerverwaltung");
}
else
{
    UpdateMainOverview("[red]Zugriff verweigert: Nur Administratoren dürfen die Benutzerverwaltung nutzen. [/]");
    UIRenderer.UIMain(MainMenuText, "Hauptmenü");
}
```

Wofür: Sperrt Menüaktionen für Nicht-Admins.

Warum: Schutz von administrativen Funktionen.

Initialer Admin beim ersten Start (Dialog)

Klasse: UserService

Methode: EnsureInitialAdmin

```
if (users == null || users.Count == 0)
{
    BodyRightManager.SetTitle("Initiale Einrichtung");
    BodyRightManager.Set("Kein Benutzer gefunden. Bitte initialen Administrator anlegen.");
    UIRenderer.Refresh(MenuSystem.StartMenu, "Startmenü");

    var admin = new User();
    admin.Username = AnsiConsole.Prompt<string>(
        new TextPrompt<string>("[bold yellow]Initialer Administrator - Benutzernamen wählen:[/]")
        .PromptStyle("green")
        .Validate(name =>
        {
            return string.IsNullOrEmpty(name) || name.Length < 3
                ? ValidationResult.Error("[red]Der Benutzername muss mindestens 3 Zeichen lang sein.[/]")
                : ValidationResult.Success();
        }
    ));

    admin.Role = UserRole.Admin;
    admin.SetPassword();
    users = new List<User> { admin };
    UserRepository.SaveUsers(users);

    TaskRepository.SaveTasks(new List<TaskItem>());

    MenuSystem.UpdateMainOverview($"[bold green]Administrator '{admin.Username}' erstellt![/]");
}
```

Wofür: Erzwingt Admin-Anlage bei jungfräulicher Datenbank.

Warum: System bleibt immer administrierbar.

JSON-Persistenz

Klasse: StorageManager<T>

Methode: Load (laden von Daten)

Methode: Save (speichern von Daten)

```
public static List<T> Load(string filePath)
{
    if (!File.Exists(filePath))
        return new List<T>();
    string jsonString = File.ReadAllText(filePath);
    return JsonSerializer.Deserialize<List<T>>(jsonString) ?? new List<T>();
}

public static void Save(string filePath, List<T> data)
{
    string jsonString = JsonSerializer.Serialize(data, new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(filePath, jsonString);
}
```

Wofür: Lädt und speichert Datenlisten als JSON auf die Platte.

Warum: Einfache Persistenz ohne Datenbank.

Session-Logik gesteuert vom User-Login

Klasse: Session

Methode: StartSession

```
while (true)
{
    if (CurrentUser == null)
    {
        BodyRightManager.SetTitle("Willkommen");
        BodyRightManager.Set(
            "Willkommen bei [bold yellow]TaskHub[/]." + Environment.NewLine +
            "Wähle 'Login' um dich anzumelden oder 'Registrierung' um einen neuen Benutzer anzulegen." +
            Environment.NewLine +
            "Benutze die Nummern im Menü zur Navigation."
        );
        UIRenderer.UIMain(MenuSystem.StartMenu, "Startmenü");
    }
    else
    {
        MenuSystem.UpdateMainOverview();
        UIRenderer.UIMain(MenuSystem.MainMenuText, "Hauptmenü");
    }
}
```

Wofür: Zeigt, welche UI der User je nach Login-Status sieht.

Warum: Steuert die Start Navigation durch das Programm und hält es am laufen.

Login-Prozess inkl. UI-Rückmeldung

Klasse: AuthManager

Methode: Login

```
var user = users.FirstOrDefault(u => u.Username.Equals(username, StringComparison.OrdinalIgnoreCase));
if (user != null && user.ValidatePassword())
{
    Session.CurrentUser = user;
    // Statistikanzeige und Begrüßung
    AnsiConsole.MarkupLine($"[green]Erfolgreich eingeloggt als {user.Username}[/]");
    return true;
}
else
{
    AnsiConsole.MarkupLine("[red]Ungültiger Benutzername oder Passwort.[/]");
    // ...
    return false;
}
```

Wofür: Überprüft Login und begrüßt/nörgelt in der UI.

Warum: Klarer Nutzer-Flow, sofortiges Feedback.

Programmablauf

Erklärung der Benutzer Oberfläche

Programmstart

Initialstart

The screenshot shows the initial state of the TaskHub application. The interface is dark-themed with yellow text. At the top, there is a header bar with three sections: 'TaskHub' on the left, 'Aufgaben Heute' with a count of 0 in the center, and 'Gesamt offen' with a count of 0 on the right. Below the header, the main area is divided into two columns. The left column contains a 'Startmenü' box with a list: '> 1. Login', '> 2. Registrierung', and '> 3. Beenden'. The right column displays a message under the heading 'Initiale Einrichtung': 'Kein Benutzer gefunden. Bitte initialen Administrator anlegen.' At the bottom of the main area, a status bar shows 'User: Nicht angemeldet' on the left and the date '10.12.2025' on the right. Below the status bar, a command prompt line reads 'Initialer Administrator - Benutzername wählen: _'.

Startet das Programm und es ist bisher kein Nutzer angelegt zwingt das Programm den Nutzer einen Initialen Administrator anzulegen.

Start mit vorhandenen Nutzern

TaskHub	Aufgaben Heute 0	Gesamt offen 0
Startmenü > 1. Login > 2. Registrierung > 3. Beenden	Willkommen Willkommen bei TaskHub. Wähle 'Login' um dich anzumelden oder 'Registrierung' um einen neuen Benutzer anzulegen. Benutze die Nummern im Menü zur Navigation.	
User: Nicht angemeldet		10.12.2025
Bitte wählen Sie eine Option: [1/2/3]: _		

Das Programm startet mit einem Menu wo der Nutzer zwischen Login und Registrierung wählen kann, innerhalb dieser Registrierung ist es nur möglich User mit der Berechtigung user anlegen.

Innerhalb dieser Registrierung können neue Nutzer nur das Zugriffsrecht user bekommen.

Hauptmenü

TaskHub	Aufgaben Heute 0	Gesamt offen 0
Hauptmenü > 1. Aufgabenverwaltung > 2. Kanban-Board > 3. Abmelden > 4. Benutzerverwaltung > 5. Beenden	Übersicht Benutzer: Admin Aufgaben heute: 0 Offene Aufgaben: 0 Gesamt Aufgaben: 0 Letzte Aktion: Eingeloggt	
User: Admin		10.12.2025
Bitte wählen Sie eine Option: [1/2/3/4/5]:		

Nach erfolgreichem Login befindet der Nutzer sich im Hauptmenü, hier kann er zwischen Aufgabenverwaltung, Kanban Board und der Benutzerverwaltung wählen ebenso ist es hier möglich sich auszuloggen oder das Programm zu beenden. Die Benutzerverwaltung kann nur von Administratoren geöffnet werden.

Aufgabenverwaltung

TaskHub	Aufgaben Heute 0	Gesamt offen 0
Aufgabenverwaltung > 1. Aufgabe erstellen > 2. Aufgaben anzeigen > 3. Aufgabe anzeigen > 4. Aufgabe bearbeiten > 5. Aufgabe löschen > 6. Zurück	Übersicht Benutzer: Admin Aufgaben heute: 0 Offene Aufgaben: 0 Gesamt Aufgaben: 0 Letzte Aktion: Aufgabenverwaltung geöffnet	
User: Admin	10.12.2025	
Bitte wählen Sie eine Option: [1/2/3/4/5/6]:		

In der Aufgabenverwaltung bekommt der Nutzer die Möglichkeit Aufgaben zu verwalten und zu sichten.

Aufgabe erstellen

TaskHub	Aufgaben Heute 0	Gesamt offen 1
Aufgabenverwaltung > 1. Aufgabe erstellen > 2. Aufgaben anzeigen > 3. Aufgabe anzeigen > 4. Aufgabe bearbeiten > 5. Aufgabe löschen > 6. Zurück	Übersicht Benutzer: Admin Aufgaben heute: 0 Offene Aufgaben: 1 Gesamt Aufgaben: 1 Letzte Aktion: Aufgabe 'TestTitel' erstellt! Fälligkeitsdatum: 12.12.2025 Erstellt: 10.12.2025 01:40	
User: Admin	10.12.2025	
Bitte wählen Sie eine Option: [1/2/3/4/5/6]: 1 Aufgaben Titel eingeben: Aufgaben Titel kurze Aufgabenbeschreibung eingeben: Aufgaben Beschreibung Fälligkeitsdatum eingeben (Format: TT.MM.JJJJ): 10.12.2025		

User eingaben finden Formular artig mit Sicht auf vorherige eingaben statt. Der Status einer Aufgabe wird beim Erstellen automatisch auf ToDo gesetzt.

Aufgaben anzeigen

TaskHub

Aufgaben Heute
1

Gesamt offen
7

Aufgabenverwaltung

- > 1. Aufgabe erstellen
- > 2. Aufgaben anzeigen
- > 3. Aufgabe anzeigen
- > 4. Aufgabe bearbeiten
- > 5. Aufgabe löschen
- > 6. Zurück

Aufgaben – Seite 1/2

test2
testtext

Fällig: 12.12.1234

test3
testtext3

Fällig: 12.12.1234

test
test

Fällig: 12.12.1234

Aufgaben Titel
Aufgaben
Beschreibung

Fällig: 10.12.2025

TestTitel
Test Beschreibung

Fällig: 12.12.2025

test4
testtesxt4

Fällig: 23.12.2345

User: Admin

10.12.2025

Wähle Aktion:

> Weiter →

Zurück zum Menü

Aufgaben werden in Seiten mit 6 aufgaben paginiert und der Nutzer bekommt die Möglichkeit mit Enter und Pfeiltasten zu blättern bzw. ins Menü zurückzukehren

Aufgabe anzeigen

Aufgabe: Aufgaben Titel

Titel

Aufgaben Titel

Beschreibung

Aufgaben Beschreibung

Erstellt

10.12.2025 01:41

Fälligkeitsdatum

10.12.2025

Status

ToDo

Zugewiesen an

Admin

ID

881eed3c-f7a0-486b-b48f-a73efc28d18d

Aufgaben werden über eine liste gewählt und daraufhin im rechten Body Bereich vollständig angezeigt

Bitte wählen Sie eine Option: [1/2/3/4/5/6]: 3

Wähle eine Aufgabe zum Anzeigen:

test2 (Fällig: 12.12.1234)

test3 (Fällig: 12.12.1234)

test (Fällig: 12.12.1234)

> Aufgaben Titel (Fällig: 10.12.2025)

TestTitel (Fällig: 12.12.2025)

test4 (Fällig: 23.12.2345)

test5 (Fällig: 12.12.5678)

← Zurück

Aufgabe bearbeiten

Auswahl der Aufgabe findet über eine Listenauswahl wie bei Aufgabe anzeigen statt

```
Bitte wählen Sie eine Option: [1/2/3/4/5/6]: 4
Neuen Aufgabentitel eingeben: (TestTitel): TestTitel
Neue Aufgabenbeschreibung eingeben: (Test Beschreibung): Test Beschreibung
Neues Fälligkeitsdatum eingeben (Format: TT.MM.JJJJ): (12.12.2025): 12.12.2025
Neuen Aufgabenstatus auswählen:

    ToDo
> InProgress
    Done
```

Für die neuen Informationen wird der Nutzer abgefragt erfolgt keine Eingabe wird der alte wert übernommen. Der Aufgabenstatus wird über eine Listenauswahl getroffen.

Aufgabe löschen

Aufgabe löschen erfolgt über eine Listenauswahl wie bei Aufgabe anzeigen.

Kanban-Board

Kanban-Board – Seite 1/1		
To Do	In Progress	Done
test2 Fällig: 12.12.1234 test3 Fällig: 12.12.1234 test Fällig: 12.12.1234 test5 Fällig: 12.12.5678	TestTitel Fällig: 12.12.2025	test4 Fällig: 23.12.2345

Das Kanban Board bietet eine simple Möglichkeit den Status seiner eigenen Aufgaben einzusehen. Bei zu vielen Einträgen wird paginiert und der Nutzer kann zwischen den Seiten blättern.

Für Aufgabe anzeigen siehe in Aufgabenverwaltung.

Aufgabe verschieben

Hier hat der Nutzer die Möglichkeit aufgaben schnell den einen neuen Status zuzuweisen.

Benutzerverwaltung

```
Letzte Aktion: Zugriff verweigert: Nur Administratoren dürfen die Benutzerverwaltung nutzen.
```

Hierauf hat nur ein Nutzeraccount mit Administratorrechten Zugriff.

TaskHub	Aufgaben Heute 0	Gesamt offen 5
Benutzerverwaltung > 1. Benutzer erstellen > 2. Benutzer anzeigen > 3. Benutzer bearbeiten > 4. Benutzer löschen > 5. Zurück	Übersicht Benutzer: Admin Aufgaben heute: 0 Offene Aufgaben: 5 Gesamt Aufgaben: 6 Letzte Aktion: Benutzerverwaltung geöffnet	
User: Admin		
10.12.2025		

Bitte wählen Sie eine Option: [1/2/3/4/5]:

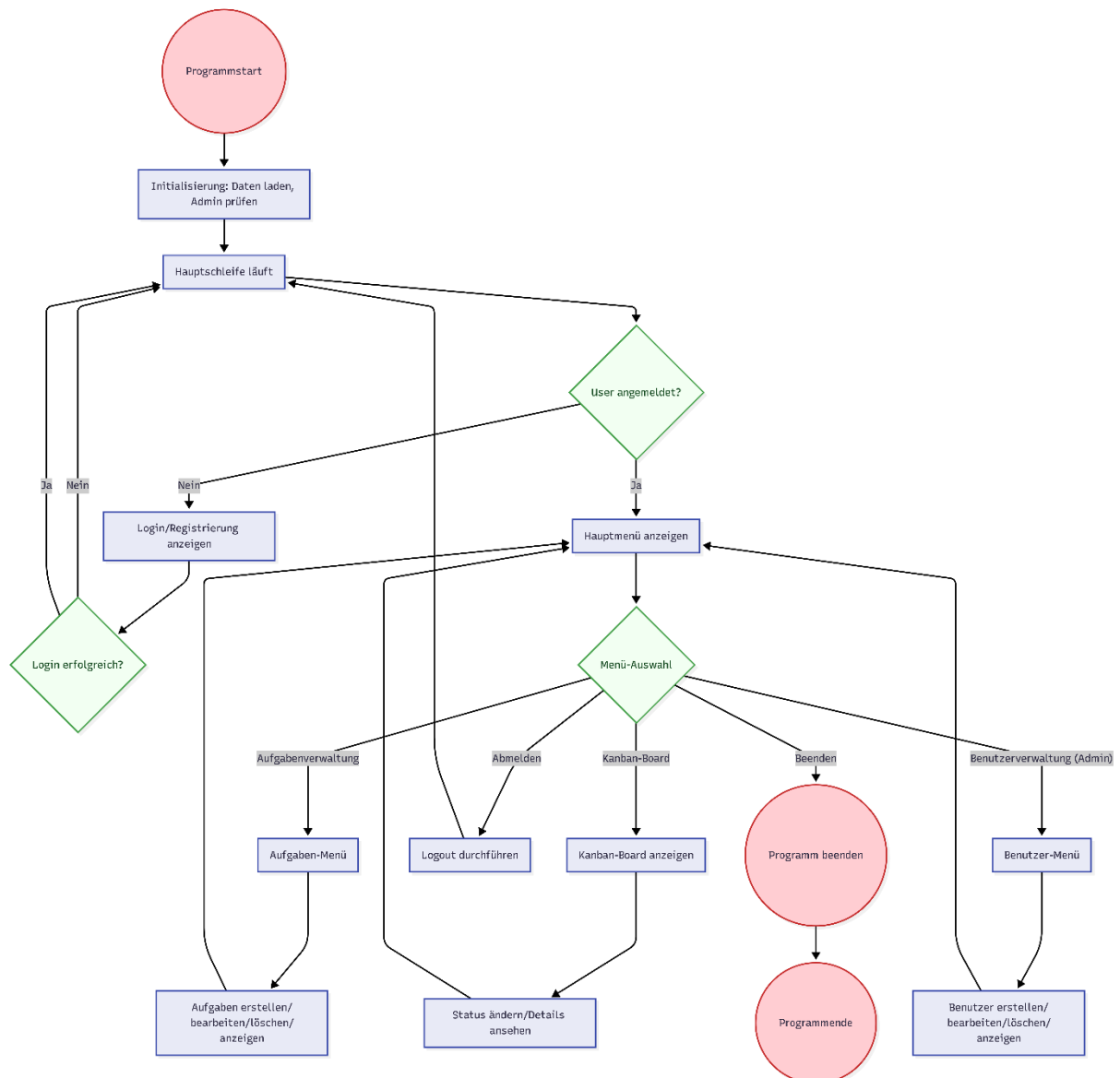
Die Menüführung folgt hier demselben Schema wie in der Aufgabenverwaltung.

Nutzer erstellen/bearbeiten

```
Bitte wählen Sie eine Option: [1/2/3/4/5]: 1
Bitte geben Sie Ihren Benutzernamen ein: testnutzer
Bitte wählen Sie Ihre Rolle:
> Admin
User
```

Hier können im Gegensatz zum Login auch Administratoren erstellt werden.

Flowchart



Das Flowchart visualisiert die wichtigsten Abläufe in der Konsolenanwendung für das Aufgaben Management Tool

Implementierung und Umsetzung

Die Umsetzung erfolgte in mehreren logisch aufeinander aufbauenden Schritten. Jeder Schritt wurde mit Datum dokumentiert ein genauer zeitverlauf mit Beschreibung der Änderungen ist über die commits im GitHub Projekt einzusehen.

Schrittweise Implementierung

Im Folgenden sind die einzelnen Arbeitsschritte der Umsetzung mit Datum und Uhrzeit (bzw. grober chronologischer Reihenfolge) dokumentiert. Die Zeitpunkte basieren auf den Commit-Meldungen. Jeder Schritt wird in vollständigen Sätzen erläutert.

02.12.2025

- Zunächst wurden die Projektdateien erstellt und das Projekt grundlegend initialisiert. Zu Beginn des Projektes wurde ein Gantt-Chart zur Projektübersicht sowie Flowcharts erstellt, um den geplanten Ablauf zu visualisieren.

03.12.2025

- Im nächsten Schritt entstand die Basis für die Benutzeroberfläche. Dazu wurde die `Spectre.Console` eingebunden und die erste Klasse `UIRenderer` erstellt. Mit der nächsten Klasse `MenuSystem` wurde die Grundlage für eine flexible Navigation im Konsolenmenü geschaffen. Um die Benutzerfreundlichkeit zu erhöhen, wurde ein Submenü eingebaut und die Darstellung mit Bildschirmlöschungen optimiert.

04.12.2025

- Die Nutzerverwaltung rückte nun in den Mittelpunkt: Es wurden erste Klassen und Hilfsmethoden für das User-Management implementiert sowie Passwort-Hashing und -Verifizierung hinzugefügt. Ein generisches JSON-Tool zum Laden und Speichern wurde entwickelt – dies bildete die Grundlage für die Wiederverwendbarkeit von Datenzugriffslogik. Das Menü für das Nutzer-Management wurde integriert und erste CRUD-Operationen (Create, Read, Update, Delete) für User realisiert.

05.12.2025

- An diesem Tag wurde das Authentifizierungssystem ausgebaut: Es kamen der AuthManager und eine Erweiterung der User-Authentifizierungslogik hinzu, ebenso wie ein Registrierungsprozess. Die Task-Verwaltung wurde durch Klassen zur Aufgabenverwaltung ergänzt und die Datenhaltung refaktorisiert. Aus der ursprünglichen „Task“-Klasse wurde „TaskItem“ und es entstand die TaskService-Klasse. Anschließend wurden Methoden des TaskService auf statische Methoden umgestellt und die Menüaufrufe daraufhin angepasst. Auch das Session-Management wurde eingeführt, um eingeloggte Sitzungen komfortabler zu verwalten. Zudem wurde nach und nach eine Panel-Struktur für die Benutzeroberfläche erarbeitet (BodyRightManager, Header/Footer-Manager) und das UI-Feedback laufend verbessert.

06.12.2025

- Im Fokus stand die Verbesserung der Übersichtlichkeit der Benutzerführung, etwa durch ein verbessertes UI-Feedback auf Login-Eingaben sowie beim Hauptmenü. Für die User-Verwaltung wurde ein paginierter Nutzer-Überblick hinzugefügt und die Panels der Benutzeroberfläche weiter optimiert.

07.12.2025

- Die Benutzerrollen-Logik wurde ausgebaut und validiert, und für Aufgabenanzeigen kam die Möglichkeit der Paginierung hinzu, um große Listen besser handhabbar zu machen. Auch die Auswahlmöglichkeiten im Menü, insbesondere bei komplexeren Menüs, wurden benutzerfreundlicher gestaltet.

08.12.2025 bis 09.12.2025

Die folgenden Tage standen vor allem im Zeichen der Task- und Userverwaltung sowie der Menüstruktur:

- Die Repositories wurden auf statische Klassen refaktoriert und die UI wurde dahingehend überarbeitet, um eine einheitliche Bedienlogik sicherzustellen.
- Es kamen Kernfunktionen wie ein Kanban-Board zur Statusdarstellung, Detailansichten für Aufgaben und ein Menüsystem für die direkte Bearbeitung von Aufgaben hinzu.
- Die Usability wurde nochmals verbessert, insbesondere durch interaktive Prompts und die automatische Erstellung eines Admin-Users beim ersten Start der Anwendung.
- Spezielles Augenmerk lag auf der Vereinheitlichung der Benennungen im Menü und den zugrunde liegenden Service-Klassen.
- Die Menülisten nutzen jetzt einheitlich `IReadOnlyList<Markup>`, und nicht mehr einzelne Listen-Typen.
- Unbenutzte using-Direktiven wurden entfernt. Es wurde ein Mechanismus implementiert, um Aufgaben eindeutige Titel zuzuweisen.
- Abschließend wurde das Feedback der Oberfläche und die Datumsformatierung konsistent und benutzerzentriert gestaltet.

Diese fortlaufenden Commits dokumentieren, wie in kleinen, nachvollziehbaren Schritten kontinuierlich neue Funktionalität ergänzt, bestehende Bestandteile verbessert und die Codebasis laufend strukturiert und erweitert wurde.

Test und Debugging-Vorgehen

- Systematische Tests: Nach jeder größeren Änderung wurden manuelle Tests der Kernpfade (Login, User CRUD, Task CRUD, MenuSystem) durchgeführt.
- Mehrere Nutzer Tests nach ersten fertigen Prototypen.

Bekannte Fehler

Nutzer eingaben die Escape-Sequenzen beinhalten können Formatierung zerstören und das Programm zum Absturz bringen.

Lösungsansätze und Designentscheidungen

- Konsolen-UI mit Spectre.Console: Die Entscheidung fiel zugunsten einer textbasierten, strukturierten UI mit Panels und Markup, um schnelle Iteration, klare Struktur und gute Lesbarkeit sicherzustellen.
- Datenhaltung: Nutzer- und Aufgabenpersistenz erfolgt über generische JSON-Utilities (Load/Save), um die Datenmodelle leichtgewichtig, portabel und versionskontrollfreundlich zu halten.
- Sicherheit: Passwort-Hashing und -Verifizierung sind früh eingeflossen, um grundlegende Sicherheitsanforderungen zu erfüllen. Die automatische Admin-Erstellung beim ersten Start vereinfacht die Initialkonfiguration.
- Usability: Paginierung für User- und Task-Listen, interaktive Prompts, konsistente Datumsformate und UI-Feedback verbessern die Nutzerführung.
- Farblayout der eingaben und wichtigen ausgaben wurden konsistent gestaltet.

Probleme, Schwierigkeiten und Überarbeitungen

- Authentifizierung und Session: Erste Versionen hatten unklare Zustandsübergänge. Lösung: AuthManager und Session-Management implementiert, Login-/Feedback-UI verbessert.
- Skalierbarkeit der Listen: Lange Listen waren unhandlich. Lösung: Paginierung in User- und Task-Ansichten eingeführt.
- Konsistenz der Benennungen: Uneinheitliche Namen in Menüs und Services erschwerten Wartung. Lösung: Refactorings der Konventionen.
- Validierung: Doppelte Task-Titel führten zu Verwechslungen. Lösung: Eindeutige Titelvalidierung im TaskService.
- Listen mit Einem Eintrag führten zum Absturz. Lösung Listen Anzahl prüfen und sonst gesonderten weg gehen

Reflektion

Reflektion der Planung

Die Gesamte Planung war sehr hilfreich und unterstützte besonders den Einstieg in die Umsetzungsphase sehr.

Da sie allerdings sehr oberflächlich war verlor sie im weiteren Verlauf stark an Bedeutung was zu häufigeren Änderungen führte.

Klassen

Die Klassen Planung erleichterte den Einstieg in die Umsetzung und konnte im Verlauf mehrfach verwendet werden.

Eine Klasse für die aktuelle Sitzung wurde nicht geplant und könnte nächstes Mal direkt eingeplant werden.

Zeitlich

Zeitliche Planung war unrealistisch was frühzeitig bemerkt wurde was durch Überstunden gepuffert wurde.

In Zukunft sollten Abschnitte vorher in kleinere Punkte unterteilt und die benötigte Gesamtzeit realistischer einschätzen zu können. Teile des Projektes streichen, um wieder in Zeitrahmen zu kommen.

Flowcharts

Die Flowcharts konnten gut zum Schreiben der Menüs genutzt werden.

Gefehlt hat zu Beginn ein Flowchart mit einem Ablauf was praktisch gewesen wäre.

Reflektion der Umsetzung

Zum Start bereitete die Umsetzung der grafischen Gestaltung der Benutzeroberfläche große Probleme da die Kenntnisse der Spectre.Console gering waren. Dies ermöglichte es allerdings einige neue Möglichkeiten zu erlernen.

Durch unbrauchbare Testdaten wurden fehlerhaft Fehler erkannt was durch unnötige Korrekturversuche zu Zeitverlusten führte.

Vorher exakt planen was für Daten und Formate für einen Test benötigt werden.

Reflektion der Dokumentation

Der Start der Dokumentation an Projekttag 1 und die Erweiterung im Laufe der Umsetzung erleichterten die endgültige Erstellung dieser Dokumentation.

Dennoch wäre es gut gewesen häufiger Notizen mit Screenshots und Zeitstempel zu versehen um damit Fehler und Lösung sowie den Verlauf besser festzuhalten damit diese einfacher in die Dokumentation eingearbeitet werden können.

Persönliche Reflektion

Ich persönlich bin mit meinem Projekt Ergebnis sowie der zugehörigen Dokumentation zufrieden und konnte meine Ziele für dieses Projekt erreichen.

Besonders die Schwierigkeit einer realistischen zeitlichen Planung ist mir in diesem Projekt deutlich geworden, weshalb ich diesem Punkt innerhalb meines nächsten Projektes genauer ausarbeiten möchte.

Quellenangaben

- Projekt-Repository: [SE-GL Abschlussprojekt](#)
- Bibliotheken:
 - Spectre.Console – <https://spectreconsole.net>
- Dokumentation/Artikel/Tutorials:
 - C#/.NET Doku – <https://learn.microsoft.com>
 - <https://stackoverflow.com>
 - <https://www.reddit.com>
 - <https://www.google.com>
- Eigene Artefakte:
 - Projektantrag und Gantt-Chart (eingefügt am 02.12.2025)
 - Flowcharts in „Projektdokumentation“
- Verwendung von KI:
 - Sammeln von Informationen über spezielle Funktionen
 - Generieren von Output texten
 - Generieren von Klassen, Methoden, Variablen -namen
 - Unterstützung zur Lösungsfindung bei Fehlern
 - Unterstützung bei der Konsistenz der Farbgebung
 - Unterstützung bei der Erstellung der Dokumentation